Apuntes del Sprint 15 · Experimentar con método: registro, validación sin fuga y elegir con honestidad

La presentación del Sprint 15 es muy esquemática: semillas, un registro, un candidato y el test una sola vez. Estos apuntes **explican el porqué de cada regla** con ejemplos numéricos pequeños, para que no sean normas que se obedecen sin entender. Léelos antes del cuaderno de seguimiento de experimentos y antes de empezar a probar modelos del proyecto. En este sprint **no hay técnicas de IA nuevas**. Ya conocéis los modelos (Sprints 7 a 10). Lo que se aprende es **cómo probarlos de forma ordenada y cómo elegir uno sin engañaros a vosotros mismos**. El enemigo no es el modelo: es **la tentación de ver lo que queremos ver**.

0. De qué va el sprint (en 1 minuto)

Cada equipo va a probar **varios modelos** sobre sus datos. Si lo hacéis a ojo, pasa esto: *«¿Cuál era el bueno? ¿Con qué ajustes? ¿Lo probó Marta o lo probé yo? ¿Era con los datos de antes de limpiar?»* Y a la semana nadie lo sabe.

El sprint se resume en **cinco hábitos**:

Hábito	En una frase
1. Reproducibilidad	Que otra persona (o vosotros en un mes) obtenga **el mismo resultado**
2. Registro	**Anotar cada prueba** en una tabla
3. Validación sin fuga	Medir con datos que el modelo **no ha podido ver**
4. Elegir con criterio	No es «el número más alto»: es **rendimiento + coste + claridad**
5. Test una sola vez	El examen final **se hace una vez**

1. Reproducibilidad: que el resultado se pueda repetir

1.1. El azar «escondido» en el código

Muchas partes de un proyecto de IA **usan azar**:

- partir los datos en entrenamiento y test al azar;
- inicializar los pesos de una red;
- elegir qué árboles forman un bosque;
- mezclar los datos antes de entrenar.

Si no controlas ese azar, **cada ejecución da un resultado distinto**. Y entonces no sabes si el modelo B es mejor que el A o si **simplemente ha tenido suerte** en esa tirada.

1.2. La semilla

Una **semilla** (*seed*) es un número que fija la secuencia de «azar» del ordenador. Con la misma semilla, el «azar» sale **igual cada vez**. (El azar del ordenador es en realidad una fórmula que parte de un número; si arrancas de ese mismo número, obtienes la misma secuencia.)

La presentación propone una función única:

```
def fijar_semillas(semilla):
    random.seed(semilla); np.random.seed(semilla)
    os.environ["PYTHONHASHSEED"] = str(semilla)
    tf.random.set_seed(semilla)         # si usas redes
```

Regla del equipo: una función, una semilla (por ejemplo, 42), **guardada en** `utilidades.py` y que **toda persona importa**. El número no importa; importa que **sea el mismo para todos**.

> **No vale cambiar la semilla hasta que salga un resultado bonito.** Eso es hacer trampa (se llama *semilla cherry-picking*). Si el resultado depende mucho de la semilla, **eso es información importante**: significa que el modelo es inestable (por eso se usa la desviación; ver apartado 4).

1.3. Las versiones también cuentan

Distintas versiones de scikit-learn, pandas o TensorFlow pueden dar **resultados distintos con el mismo código**. Por eso se anotan las versiones (en `requirements.txt` o en el registro).

Comprobación de reproducibilidad (la regla de oro): ¿el notebook **se ejecuta de arriba abajo** en una máquina limpia y da lo mismo? Si no, **no es reproducible**. En la rúbrica es el criterio F.

---

2. El registro de experimentos: una fila por prueba

2.1. Para qué sirve

Es una **tabla** (un CSV en el repositorio: `experimentos/registro.csv`) donde **cada prueba** deja una fila. Se abre con Excel y **Git guarda su historia**.

Columna	Qué anota	Ejemplo
id	Número de prueba	7
fecha	Cuándo	2026-12-02
persona	Quién la hizo	Marta
modelo	Qué modelo	RandomForest
hiperparámetros	Ajustes	`max_depth=6, n_estimators=200`
partición	Con qué datos se midió	`temporal_2022-2024`
métrica (media)	Resultado de la validación	0,71
desviación	Cuánto varía	0,02
tiempo	Cuánto tardó (s)	14
notas	Lo que se aprendió	«Empeora al subir profundidad: sobreajuste»

> **La columna de notas es la más valiosa.** Los números los puede regenerar el código. **Lo que has pensado, no.**

2.2. Un registro bien hecho

Un ejemplo de seis filas (cifras inventadas):

id	persona	modelo	ajustes	F1 (media ± desv.)	tiempo	notas
1	Ana	Baseline (clase más frecuente)	—	0,00 ± 0,00	0,1 s	Es el listón: el F1 es 0 porque nunca predice la clase rara
2	Ana	Regresión logística	`C=1`	0,58 ± 0,02	0,3 s	Modelo simple de referencia
3	Marta	Bosque aleatorio	`max_depth=3`	0,64 ± 0,02	9 s	Mejora
4	Marta	Bosque aleatorio	`max_depth=10`	0,66 ± 0,03	21 s	Apenas mejora y tarda el doble
5	Luis	Boosting	`lr=0.1`	0,70 ± 0,02	14 s	El mejor hasta ahora
6	Luis	Red neuronal	`64-32`	0,71 ± 0,05	210 s	**Casi igual, más inestable y 15 veces más lenta**

Fíjate en que **de este registro se lee una historia**: se ve **qué se probó, en qué orden y por qué se decidió**. Esa historia es la que se defiende en la review y en el Sprint 17.

2.3. Reglas del registro

- **Una fila por prueba**, también las que **salen mal** (un «esto no funciona» **ahorra a otros** repetirlo).
- **Todas las personas del equipo** tienen **al menos una fila** (si solo hay una persona escribiendo, la nota individual lo refleja).
- **≥ 12 filas** recomendadas.
- Se marcan las pruebas con **código generado por IA** (el reto lo pide).
- **Si el cómputo es limitado** (GPU lenta, API de pago), se usan subconjuntos y **se anota el coste**.

> **¿Y MLflow?** Es una herramienta profesional que hace lo mismo con una interfaz web. **Es opcional.** El CSV **cumple la rúbrica**.

---

3. Validación sin fuga de datos

3.1. La idea de fondo

Un modelo se evalúa con **datos que no ha visto al aprender**, igual que un examen no puede tener las preguntas que ya habías visto resueltas (Sprints 6 y 7).

Fuga de datos (*data leakage*) = **que información que no estaría disponible en el momento real de decidir se cuele en el entrenamiento o en la elección del modelo.** El resultado parece buenísimo… **y es falso**.

3.2. Un ejemplo clásico: el «escalado antes de partir»

Imagina estos cinco valores de una variable: `10, 12, 11, 13, 50`. El último (50) es un caso extremo, y está **en el test**.

Cómo se calcula la media para «escalar»	Media
Con todos los datos (antes de partir)	**19,2**
Solo con el entrenamiento (`10, 12, 11, 13`)	**11,5**

Con la primera forma, **el entrenamiento ya «sabe» que existe un 50** (su media sube de 11,5 a 19,2): **el test ha dejado su huella en el entrenamiento**. Es una fuga **sutil**, no se nota a simple vista, y **hace que la evaluación sea más optimista**.

Cómo evitarla: metiendo el escalado, la imputación y la codificación **dentro de un `Pipeline`**, que **aprende esas cosas solo con los datos de entrenamiento** de cada partición (esto lo visteis en los Sprints 6 y 7).

3.3. Las cuatro fugas típicas del proyecto

Fuga	Ejemplo	Cómo se arregla
Una variable del futuro	Predecir cancelaciones usando «fecha de devolución del dinero» (que solo existe **si** se canceló)	Pregunta: **«¿conocería este dato en el momento de decidir?»**
Preparar antes de partir	Escalar, imputar o codificar con todos los datos	**`Pipeline`**
Partición que mezcla el tiempo	En una serie, entrenar con meses futuros para predecir los pasados	**Partición temporal**: entrenar con el pasado, evaluar con lo posterior
Fotos del mismo objeto en train y test	Tres fotos de la misma hoja: una en entrenamiento y otra en test	**Partir por objeto o por parcela**, no por foto

Por qué lo de las fotos importa: si dos fotos de la misma hoja están una en train y otra en test, el modelo **reconoce la hoja concreta** (memoriza), no la enfermedad. Saldrá «genial»… **y fallará con hojas nuevas**.

3.4. La señal de alarma: un resultado «demasiado bueno»

Si en un problema difícil (predecir cancelaciones, demanda, enfermedades) tu modelo da un **99 %**, **lo más probable es que haya fuga**, no que seas un genio. **Sospecha, no celebres.**

3.5. Lista de comprobación (la del reto)

Antes de elegir modelo, responde **sí / no**:

1. ¿Todas las variables **se conocerían en el momento de decidir**?
2. ¿Escalado, imputación y codificación están **dentro del `Pipeline`**?
3. ¿La partición respeta el **tiempo** (series) o los **grupos** (fotos del mismo objeto)?
4. ¿Se ha ajustado **algo** mirando el test? (Si sí: hay que rehacer la evaluación.)
5. ¿El resultado es **demasiado bueno para ser verdad**?

---

4. Validación cruzada y «media ± desviación»

4.1. Por qué una sola medición no basta

Si divides los datos **una vez** y mides, el resultado depende de **qué filas han caído en cada lado**. Con otra partición, saldría otro número. **¿Cuánto fiarse de un solo número?**

Validación cruzada de k pliegues (Sprint 7): se parte el entrenamiento en *k* trozos (por ejemplo 5). Se entrena **5 veces**, cada vez dejando **un trozo fuera para medir** (con 100 filas: 80 para entrenar y 20 para medir, y se va rotando). Resultado: **5 medidas**, de las que se calcula:

- **Media**: el resultado «típico».
- **Desviación**: **cuánto varía** entre pliegues (cuanto mayor, **más inestable** o más incierto el resultado).

> **En series temporales, los pliegues respetan el orden** (siempre se entrena con lo anterior y se mide con lo posterior), **no se reparten al azar**.

4.2. Ejemplo numérico: ¿cuál de los dos es mejor?

Dos modelos, 5 pliegues, F1 en cada pliegue:

Pliegue	Modelo A	Modelo B
1	0,69	0,74
2	0,71	0,66
3	0,70	0,77
4	0,72	0,65
5	0,68	0,73
Media	**0,70**	**0,71**
Desviación	**0,02**	**0,05**

- **B tiene la media más alta** (0,71 frente a 0,70)… pero **varía el triple** (0,05 frente a 0,02). A veces saca 0,77 y a veces 0,65.
- La diferencia de medias (**0,01**) es **mucho más pequeña que la variación** del propio modelo B. **No hay base para decir que B es mejor.** Probablemente es **ruido**.

4.3. Regla práctica del «ruido»

Situación	Qué concluir
Diferencia **mucho mayor** que las desviaciones	Probablemente **hay mejora real**
Diferencia **del tamaño de las desviaciones** o menor	**Puede ser ruido**: no declares ganador
Dos modelos prácticamente empatados	**Elige el más simple, más rápido o más explicable**

> La pregunta que os harán los demás equipos en la review: **«¿Cómo sabéis que la diferencia no es ruido?»** Con media ± desviación, tenéis una respuesta.

5. Elegir el candidato: no es «el número más alto»

5.1. Tres cosas a la vez

Dimensión	Pregunta
Rendimiento	¿Cuánto mejora el baseline? ¿Es una mejora clara o ruido?
Coste	¿Cuánto tarda en entrenar y en predecir? ¿Necesita GPU? ¿Es caro?
Claridad	¿Podemos **explicar** por qué decide lo que decide?

5.2. Un ejemplo de decisión

Modelo	F1 (media ± desv.)	Tiempo	Explicable
Baseline	0,00 ± 0,00	0,1 s	—
Regresión logística	0,58 ± 0,02	0,3 s	Mucho
Boosting	**0,70 ± 0,02**	**14 s**	**Bastante**

Modelo	F1 (media ± desv.)	Tiempo	Explicable
Red neuronal	0,71 ± 0,05	210 s	Poco

Razonamiento:

- La red es **0,01** mejor (**1,4 %** en relativo: (0,71 − 0,70)/0,70), pero con **desviación 2,5 veces mayor** y **15 veces más lenta** (210 s / 14 s = 15) y **menos explicable**.
- La diferencia puede ser ruido; **no compensa el coste**.
- **Elegimos el boosting**, y **lo decimos explícitamente**: *«es la mejor relación rendimiento-coste-claridad»*.

> **Elegir un modelo «peor» en el número, pero más simple, es una decisión sensata y valorada.** Lo que **no** vale es elegirlo **sin haberlo razonado**.

5.3. La hipótesis de por qué gana (nivel «Excelente»)

La rúbrica premia explicar **por qué** gana. Fórmula útil:

> *«Gana el [modelo] porque [característica de los datos] y [característica del modelo].»*

Ejemplos:

Hipótesis	Por qué tiene sentido
«Gana el boosting porque la **antelación y el canal interactúan** (reservar con mucha antelación por web cancela más) y los árboles capturan esas interacciones; la regresión lineal, no»	Conecta con lo visto en el Sprint 8
«Gana la red con transfer learning porque **solo tenemos 800 fotos** y reaprovecha lo ya aprendido»	Conecta con el Sprint 11
«El modelo simple empata porque **la relación es casi lineal**»	Una conclusión honesta

Es una hipótesis: se puede comprobar después (por ejemplo, con la importancia de variables o con SHAP). No hace falta que sea cierta al 100 %, pero **tiene que estar razonada**.

---

6. El test: una sola vez

6.1. Por qué «una sola vez»

El **test** son **los datos que se reservan para el examen final**. Si lo usas **para elegir**, deja de ser un examen sorpresa: **ha influido en tu decisión**, aunque sea indirectamente.

6.2. Un experimento mental que lo demuestra

Imagina que **200 modelos son puro azar** (adivinan lanzando una moneda; su verdadera exactitud es 50 %). Los evalúas todos en el **mismo test de 100 ejemplos** y te quedas con el mejor.

Resultado de la simulación (semilla 1): el «mejor» de los 200 acierta **69 de 100**. **Un 69 % con un modelo que adivina al azar.** Parece un resultado estupendo… y **es pura casualidad**: con tantos intentos, alguno saldrá afortunado.

> **Esa es la razón de la regla «nada de probar 200 combinaciones y quedarse con la mejor».** Cuantas más cosas pruebas **mirando el test**, más probable es que lo que ves **sea suerte**.

Por eso:

Se hace en…	Con qué datos
Probar y comparar modelos	**Validación cruzada del entrenamiento**
Elegir el candidato	**Esa misma validación**
Evaluar el elegido	**El test, UNA vez**

6.3. ¿Y si el resultado en test es peor?

Es **normal** que sea algo peor que en validación (por ejemplo, 0,70 en validación → 0,69 en test). Si es **mucho** peor, algo pasa (fuga en la validación, sobreajuste a la validación).

Lo que NO se hace: volver a elegir mirando el test. **Se anota, se explica y se sigue.** Un resultado honesto y mediocre **vale más** que uno bonito conseguido haciendo trampa.

6.4. Guardar el modelo

Se guarda en la carpeta `modelo/` junto con un `metadatos.json` (versión, fecha, métricas, variables usadas), para que **el demostrador del Sprint 16 cargue exactamente este modelo**.

---

7. Una primera explicación del modelo

Un número de rendimiento no explica **por qué** decide lo que decide. En este sprint se prepara una **primera explicación** (se profundiza en el Sprint 16):

Tipo de dato	Técnica	Qué te dice
Tabular	**Importancia de variables** o **SHAP**	Qué variables pesan más y en qué dirección
Imágenes	**Grad-CAM**	**Qué zona de la imagen** miró el modelo

Cómo leerla con criterio: pregúntate **«¿tiene sentido para el negocio?»** Si el modelo de cancelaciones dice que lo más importante es el **número de reserva** (un código sin significado), **huele a fuga** o a casualidad.

---

8. El trabajo en equipo este sprint

- **Roles nuevos** (se rotan; Sprint 0).
- **Cada persona anota al menos una fila** del registro.
- **Una única partición común, guardada**: **no cada uno la suya**. Si cada persona usa una partición distinta, **no se pueden comparar los resultados**.
- **Tutoría de 15 minutos**: contad **qué comparación hacéis y qué fuga podría haber**.
- **Retro con una acción concreta** del sprint anterior.

---

9. Errores frecuentes (y cómo evitarlos)

Error	Por qué es un problema	Cómo evitarlo
Elegir mirando el test	El test deja de ser un examen	Elegid con validación; el test, una vez
Resultados «increíbles» (99 %)	Casi seguro, fuga	Pasad la lista del apartado 3.5

Error	Por qué es un problema	Cómo evitarlo
Cada persona con su partición	Resultados que no se pueden comparar	Una partición común
Registro de 3 filas de una persona	No demuestra trabajo en equipo ni método	Todas las personas anotan
Mejor número = mejor modelo	Ignora el coste y la explicación	Tabla con rendimiento, tiempo y explicabilidad
No apuntar los fallos	Se repiten errores	Una fila también para lo que sale mal
Cambiar la semilla hasta que sale bien	Es trampa, aunque sea sin querer	Una semilla fija para todos
Mezclar fotos del mismo objeto	El modelo memoriza	Partir por objeto o por parcela

10. Cómo usar la IA en este sprint (y cómo comprobarla)

Para qué	Qué comprobar
Entender un modelo	Que la explicación **encaja con vuestros datos**
Depurar un error	Que entendéis **la causa**, no solo copiáis el arreglo
Buscar fugas en vuestro pipeline	La IA puede **pasar por alto** una fuga **o ver una que no existe**: verificad con la lista del apartado 3.5
Interpretar SHAP	Contrastadlo **con el sentido del negocio**

Y marcad en el registro las pruebas hechas con código generado por IA.

11. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Semillas y versiones	1	Cuaderno `S15_01_seguimiento_experimentos` (ejercicio 1)
El registro de experimentos	2	Cuaderno, ejercicios 2 a 4; Paso 1 del reto
Fuga de datos	3	Cuaderno (filas «Fuga») y Paso 3 del reto
Media ± desviación y ruido	4	Paso 4 del reto
Elegir candidato y justificarlo	5	Tabla de justificación del reto
El test una sola vez	6	Cuaderno, ejercicio 5; Paso 5 del reto
Explicar el modelo	7	Paso 5 del reto

12. Chuleta de vocabulario

Palabra	En lengua normal
Semilla	Número que fija el azar para repetir resultados
Registro de experimentos	Tabla con una fila por prueba
Hiperparámetro	Ajuste que se fija antes de entrenar

Palabra	En lengua normal
Fuga de datos	Información «del futuro» o del test que se cuela en el entrenamiento
Pipeline	Cadena de preparación + modelo que aprende solo del entrenamiento
Validación cruzada	Medir varias veces rotando qué parte se deja fuera
Media ± desviación	Resultado típico y cuánto varía
Ruido	Variación por casualidad
Candidato	El modelo que proponéis
Sobreajuste	Memorizar el entrenamiento y fallar con datos nuevos
Coste / beneficio	Lo que se gana frente a lo que cuesta
SHAP / Grad-CAM	Técnicas para ver qué influye en una decisión

13. Cómo estudiar estos apuntes

1. **Antes de empezar a probar modelos**, leed las secciones 1 a 3: son los **hábitos** que luego es difícil adquirir.
2. **Antes de elegir el candidato**, repasad las secciones 4 y 5.
3. **Antes de tocar el test**, releed la sección 6. Si hay duda, **no lo toquéis**.
4. **Regla final:** si no podéis decir **cómo sabéis que la diferencia entre dos modelos no es ruido** y **cuándo se usó el test**, **el sprint no está terminado**.